Max Flow

Yufei Tao

Department of Computer Science and Engineering
Chinese University of Hong Kong

The Max Flow Problem

$G = (V, E)$: a directed simple graph.

Given a vertex $v \in V$, we will use

- $\text{IN}(v)$ to denote the set of in-neighbors of v;
- $\text{OUT}(v)$ to denote the set of out-neighbors of v.

G has no "isolated" vertices v with $\text{IN}(v) = \text{OUT}(v) = \emptyset$.

s: a vertex in G called the **source vertex**, which satisfies $\text{IN}(s) = \emptyset$
t: a vertex in G called the **sink vertex**, which satisfies $\text{OUT}(t) = \emptyset$

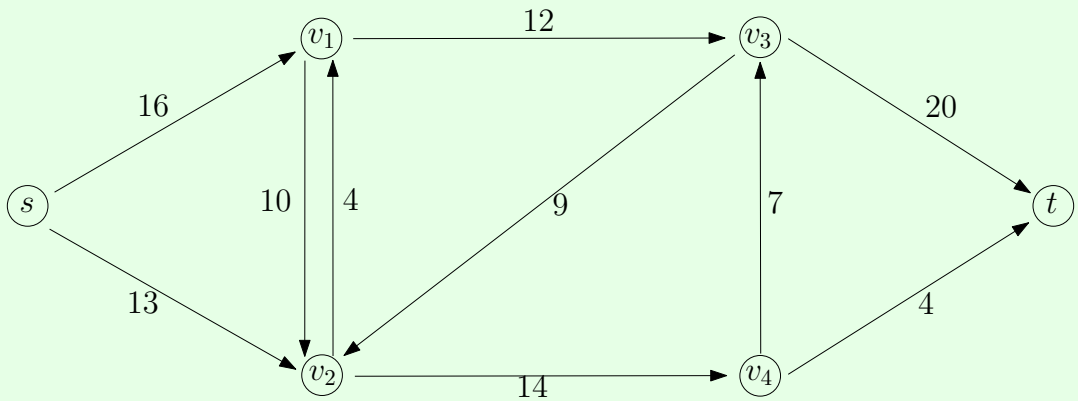
The Max Flow Problem

$\mathbb{N}_{\geq 0}$: The set of non-negative integers

We are given a capacity function $c : V \times V \to \mathbb{N}_{\geq 0}$ satisfying:

- $c(u, v) = 0$ if $(u, v) \notin E$.

Example:



The numbers indicate the edge capacities.

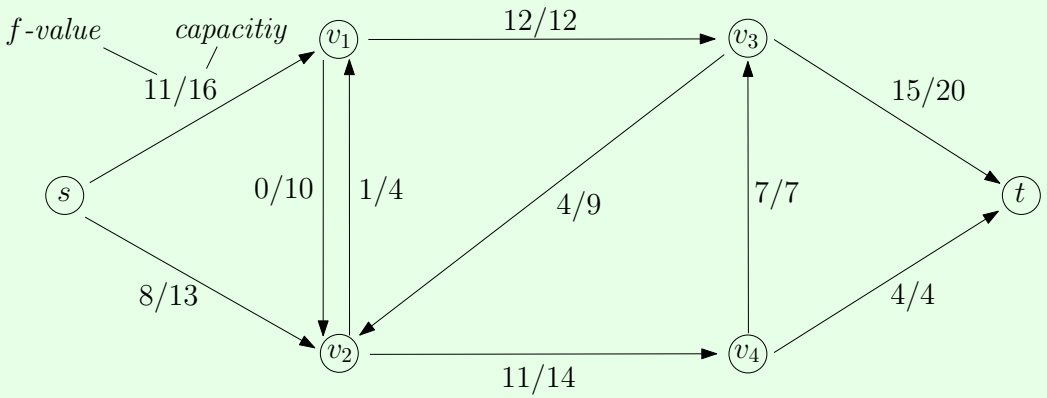
The Max Flow Problem

A **flow** is a function $f : V \times V \to \mathbb{N}_{\geq 0}$ satisfying:

- (**capacity constraint**) $f(u, v) \leq c(u, v)$ for any $u \in V$ and $v \in V$;
- (**conservation constraint**) for each vertex $v \in V \setminus \{s, t\}$

$$\sum_{u \in \mathsf{IN}(v)} f(u, v) = \sum_{w \in \mathsf{OUT}(v)} f(v, w).$$

Example:



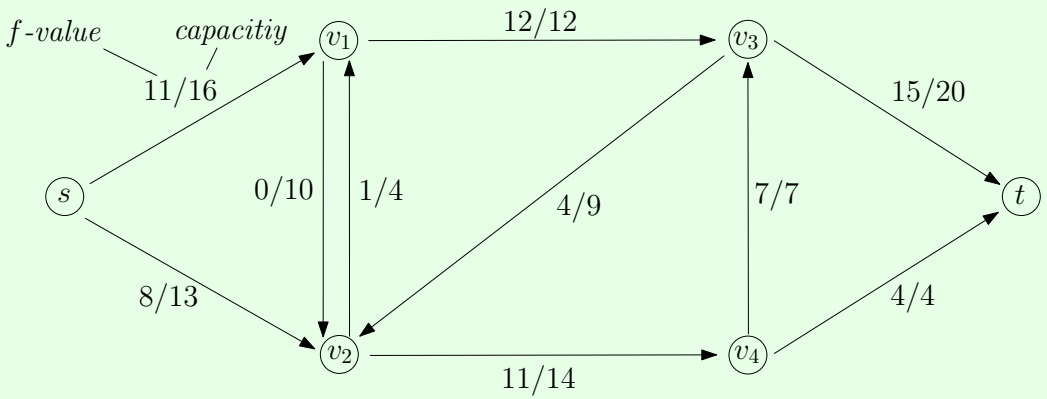
For each edge, we represent its f-value x and c-value y in the form of x/y.

The Max Flow Problem

Given a flow f, we define its **value** as:

$$|f| = \sum_{u \in \text{OUT}(s)} f(s, u).$$

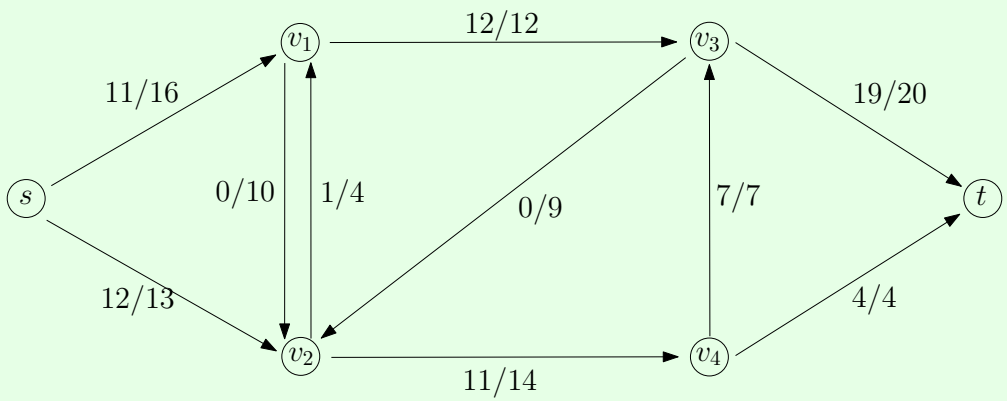
Example:



This flow has value 19.

Goal of the Max Flow Problem:
Find a flow f with the maximum value $|f|$.

Example:



A max flow with value 23.

Lemma 1: For any flow f, it holds that

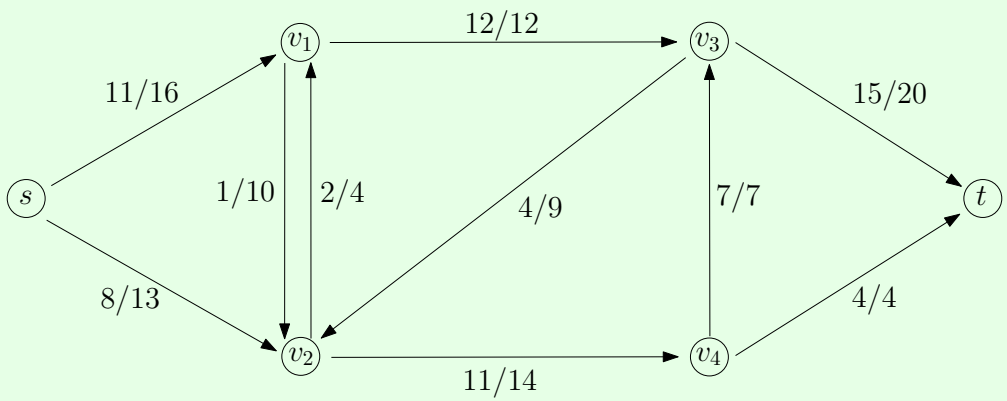
$$|f| = \sum_{u \in \text{IN}(t)} f(u, t).$$

The proof is left as an exercise.

A flow f is **clean** if it has the property that:
for any $u \in V$ and $v \in V$, if $f(u, v) > 0$, then $f(v, u) = 0$.

Otherwise, the flow is **dirty**.

Example: A dirty flow:



The lemma below suggests that we can focus on clean flows.

Lemma 2: The maximum value of all flows equals the maximum value of all clean flows.

The proof is left as an exercise.

We will focus on finding a **clean** flow with the maximum value.

Given a clean flow f, we can define the **residual function** $r : V \times V \to \mathbb{N}_{\geq 0}$ by setting

$$r(u, v) \quad = \quad c(u, v) - f(u, v) + f(v, u)$$

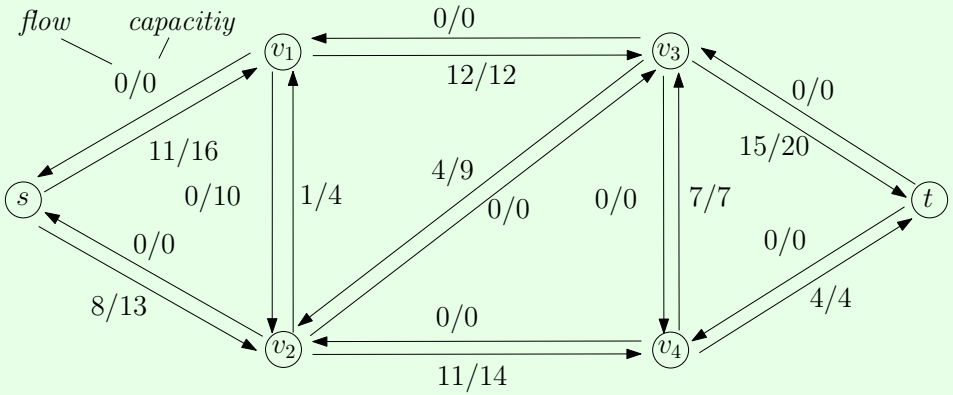
for any $u, v \in V$.

Residual graph: A directed graph $G_f = (V, E_f)$ where

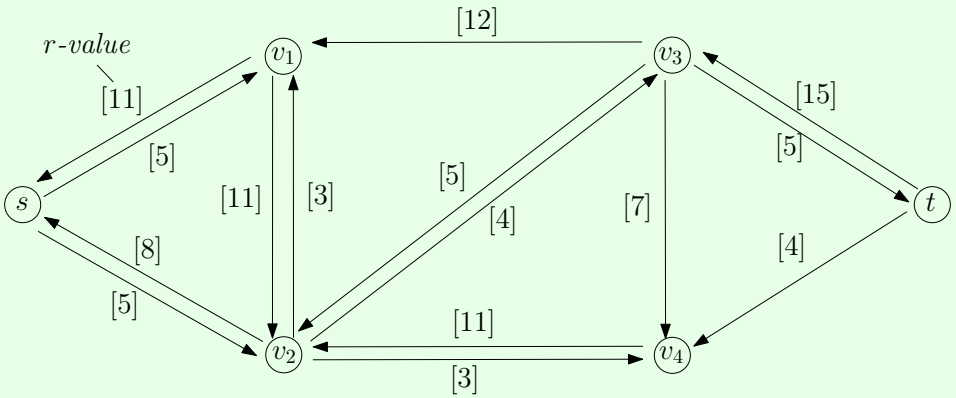
$$E_f = \{(u, v) \in V \times V \mid r(u, v) > 0\}.$$

Example:

A flow f:

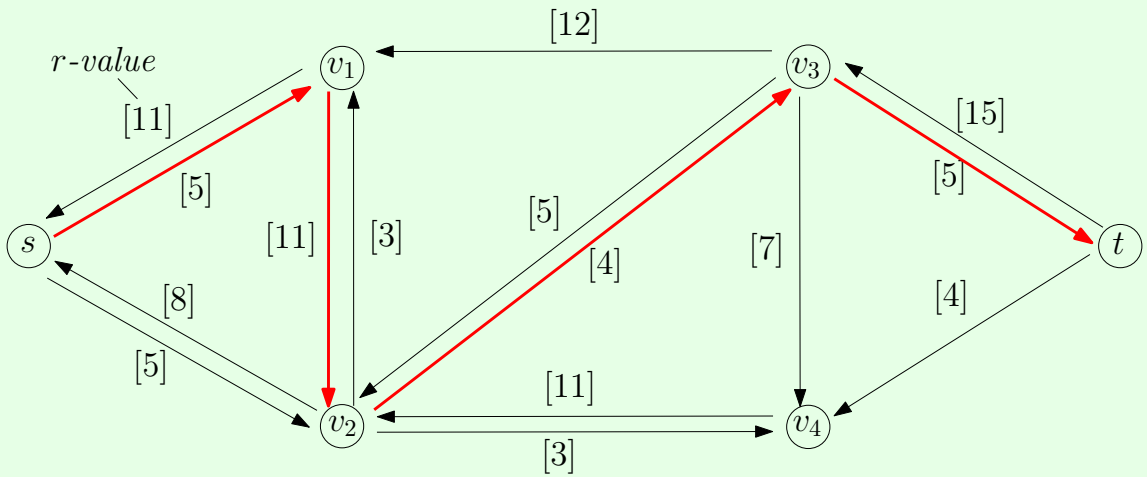


The residual graph G_f:



An **augmenting path** is a simple path π from s to t in G_F.
The **capacity** of π is the smallest r-value of the edges on π.

Example:
The red path is an augmenting path with capacity 4.



Given G_F, we can find an augmenting π in $O(|E|)$ time.

How to do so with BFS/DFS?

We will show how to use an augmenting path to obtain a better flow. Identify an arbitrary augmenting path π, and let Δ be the capacity of π.

Initialize $f' = f$. For each edge (u, v) on π, update both directions using the original values of f:

- if $f(v, u) \geq \Delta$, set

$$f'(v, u) = f(v, u) - \Delta, \text{and } f'(u, v) = 0;$$

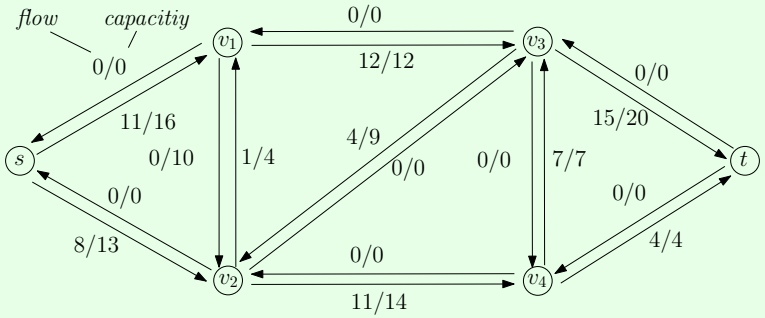
- otherwise, set

$$f'(v, u) = 0, \text{and } f'(u, v) = f(u, v) + \Delta - f(v, u).$$

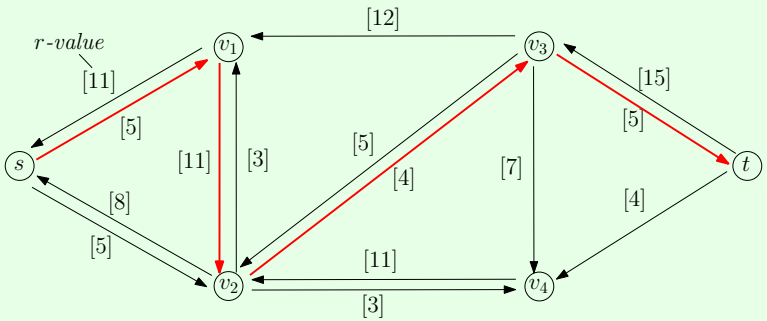
Lemma 3: The function f' thus constructed is a clean flow with $|f'| = |f| + \Delta$.

The proof is left as an exercise.

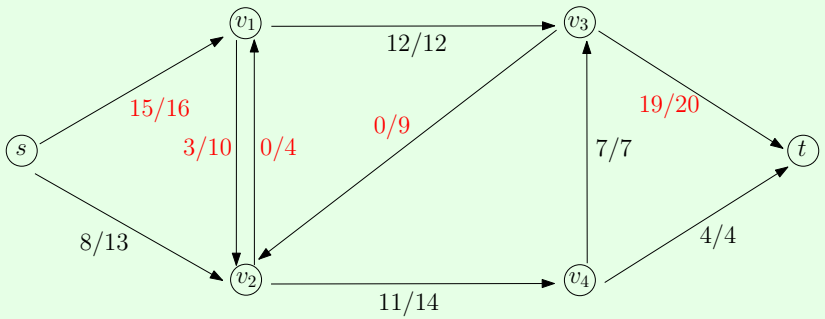
Example: A flow f:



An augmenting path π in the residual graph G_f:



The new flow f':



The Ford-Fulkerson Algorithm

1. $f \leftarrow$ the zero flow
2. **do**
3. $\quad G_f \leftarrow$ the residual graph decided by f
4. $\quad$ find an augmenting path π
5. $\quad$ **if** π does not exist **then** return f
6. $\quad f' \leftarrow$ the new clean flow constructed from f and π
7. $\quad f \leftarrow f'$

The rest of the slides will prove

- the algorithm is correct;
- its running time is $O(|E|(M+1))$, where M is the maximum flow value.

Let S and T be subsets of V satisfying

- $S \cup T = V$ and $S \cap T = \emptyset$;
- $s \in S$ and $t \in T$.

We say that they make an S-T **cut**.

Lemma 4: For any flow f and any S-T cut,

$$|f| = \Big(\sum_{u \in S, v \in T} f(u,v) \Big) - \Big(\sum_{u \in T, v \in S} f(u,v) \Big)$$

The proof is left as an exercise.

We define the **capacity** of an S-T cut as

$$\text{cap}(S, T) = \sum_{u \in S, v \in T} c(u, v).$$

The following can be obtained immediately from the previous lemma.

Corollary 5: $|f| \leq \text{cap}(S, T)$ holds for any flow f and any S-T cut.

The Max-Flow Min-Cut Theorem:
Let f be an arbitrary clean flow. The following statements are equivalent:

1. f has the largest value among all flows.
2. G_f (the residual graph decided by f) has no augmenting paths.
3. There exists an S-T cut such that $|f| = \text{cap}(S, T)$.

Proof. We will prove

- statement 1 $\Rightarrow$ statement 2
- statement 2 $\Rightarrow$ statement 3
- statement 3 $\Rightarrow$ statement 1

which imply that all three statements are equivalent.

Proof of $1 \Rightarrow 2$. If f has the maximum value, then G_f has no augmenting paths. Otherwise, augmenting along such a path yields a clean flow with a strictly larger value, by Lemma 3.

Proof of $3 \Rightarrow 1$. If f does not have the maximum value, then let f' be a flow with $|f'| > |f|$. But then $|f'| > |f| = \text{cap}(S, T)$, contradicting Corollary 5.

Proof of $2 \Rightarrow 3$. Define

$$\begin{aligned} S &= \{v \in V \mid s \text{ can reach } v \text{ in } G_f\} \\ T &= V \setminus S. \end{aligned}$$

Clearly, $s \in S$. As G_f has no augmenting paths, we know $t \in T$.

Consider any $u \in S$ and any $v \in T$. The edge (u, v) cannot exist in G_f (otherwise, s would be able to reach v, contradicting $v \in T$).

Hence, the residual capacity satisfies

$$0 = r(u, v) = \underbrace{c(u, v) - f(u, v)}_{\geq 0} + \underbrace{f(v, u)}_{\geq 0}.$$

Both terms must be zero. Thus, $f(u, v) = c(u, v)$ and $f(v, u) = 0$.

It thus follows that

$$\begin{aligned} |f| &= \Big(\sum_{u\in S, v\in T} f(u,v)\Big) - \Big(\sum_{u\in T, v\in S} f(u,v)\Big) \quad \text{(by Lemma 4)} \\ &= \sum_{u\in S, v\in T} c(u,v) - 0 \\ &= \text{cap}(S,T). \end{aligned}$$

□

By Corollary 5, every cut has capacity at least $|f|$. Thus, this cut has **minimum capacity**: the maximum flow value equals the minimum cut capacity.

The correctness of the Ford-Fulkerson algorithm follows from the max-flow min-cut theorem.

Let M be the maximum flow value. Starting from zero flow, all residual capacities and augmentation amounts are integers. Each augmentation increases the flow value by **at least** 1.

There are at most M augmentations and one final unsuccessful search. Each iteration takes $O(|E|)$ time: the residual graph has at most $2|E|$ edges, and $|V| = O(|E|)$ because there are no isolated vertices.

The total running time is $O(|E|(M+1))$, or $O(|E|M)$ when $M \geq 1$.

Remark: If we use the BFS algorithm to find an augmenting path in each iteration, it is possible to show that the Ford-Fulkerson algorithm finishes in $O(|V||E|^2)$ time. The details are not required in this course.